Design Document: Agentic AI Mini-Project

Ariadne — Sericulture Supply Chain Resilience through Agentic Triage and Dynamic Rerouting

Course: Agentic AI (26CD7PEAAI) | Domain: Precision Sericulture & Agricultural Supply Chains | R2: 24 October 2026

Team 14: Arunank Shankar (1BM23CD007) · Deepthi M Sharma (1BM23CD015) · Pavithra Manikandan (1BM23CD043) · Prajwal Kiran (1BM23CD044)

1 Project Overview

Ariadne addresses disruption in the silk supply chain, from mulberry cultivation to the sale of structural thread. Disruption can start at any tier. A disease or pest outbreak reduces the quality and quantity of a farm's mulberry leaves and silkworms, which hits the cocoon supply to reelers. A reeler may be unable to produce enough yarn, or may be delayed, which hits his spinners. A spinner may face higher demand and need more yarn from reelers. Each actor reports such an incident or request through a role-aware chatbot. Ariadne combines multi-modal disease triage, agentic retrieval over sericulture knowledge, severity–urgency assessment and inventory-aware rerouting in the direction the problem requires. Every deal or reroute is finalised only after human approval.

Actors. The *Farmer* (moriculture and rearing) grows mulberry leaves and silkworms and sells raw cocoons. The *Reeler* (boiling and reeling) sells raw silk yarn as hanks or bales. The *Throwster/Spinner* (twisting and preparing yarn) sells structural thread. *Warehouses* serve the Reeler and Spinner respectively. The Farmer is responsible for quality and quantity, the Reeler for a correct supply of yarn, and the Spinner for receiving it on time.

Role workflows.

- **Farmer:** uploads a photo and gets a disease result with severity and urgency, mitigation methods, and advice on whether to escalate to the reeler. Sees collaborating reelers (and how many an issue affects), available and sellable stock, and the money balance (HITL on deals).
- **Reeler:** checks stock, time left until degradation and how to raise falling stock. Can report an incident (too little yarn from a factory issue or low cocoon supply, or a delivery delay). Ariadne rates it, identifies the affected spinners and proposes resolutions such as re-allocating remaining yarn or finding an alternative source, leading to rerouting and a deal. Also sees collaborating spinners, safety stock, sellable stock and balance.
- **Spinner:** sets a target quantity or reports higher demand and checks whether available reelers can meet it. If not, Ariadne searches alternative reelers, plans routes and prepares a deal. Also sees safety stock, his reelers and balance.

Data visibility. A mulberry farmer sees only his own farm; a silkworm farmer sees the mulberry farmer supplying him and his own farm; a warehouse owner sees the silkworm farmers linked to his warehouse and his own warehouse.

2 Architecture Soundness

2.1 Architecture Overview and Incident Scenarios

Figure 1 shows three cooperating layers: a data and knowledge layer, a LangGraph orchestration layer, and a supply-chain recovery pipeline, with supporting services across all three. The orchestration layer has three entry points handled by one Triage Supervisor and four specialist agents. Whatever the entry point, a supply impact flows into one shared recovery pipeline; only the search direction differs (upstream for suppliers, downstream for buyers). Table 1 shows what each actor can trigger.

Table 1. Incident scenarios handled by the shared workflow.

Entry point	Typical trigger	Specialist and assessment	Rerouting and deal
Farmer incident	Leaf or silkworm image, or a chat query about a suspected disease or pest.	Mulberry or Silkworm Agent with Agentic RAG: diagnosis, severity and urgency, mitigation, escalate or not.	Affected reelers identified; if cocoon supply is at risk, alternative farmers are searched and routed; farmer–reeler deal approved.
Reeler incident	Too little yarn (factory issue or low cocoon supply) or a delivery delay, reported via chatbot.	Reeler Agent with Agentic RAG and stock figures: severity and urgency, time until stock degrades, resolution options.	Affected spinners identified; yarn re-allocated, or alternative reelers or cocoon sources searched and routed; reeler–spinner deal approved.
Spinner request	Higher demand, or a target the current reelers may not meet.	Spinner Agent with safety-stock and capacity calculations: gap against target, urgency.	Alternative reelers searched and routed for the shortfall; reeler–spinner deal approved.

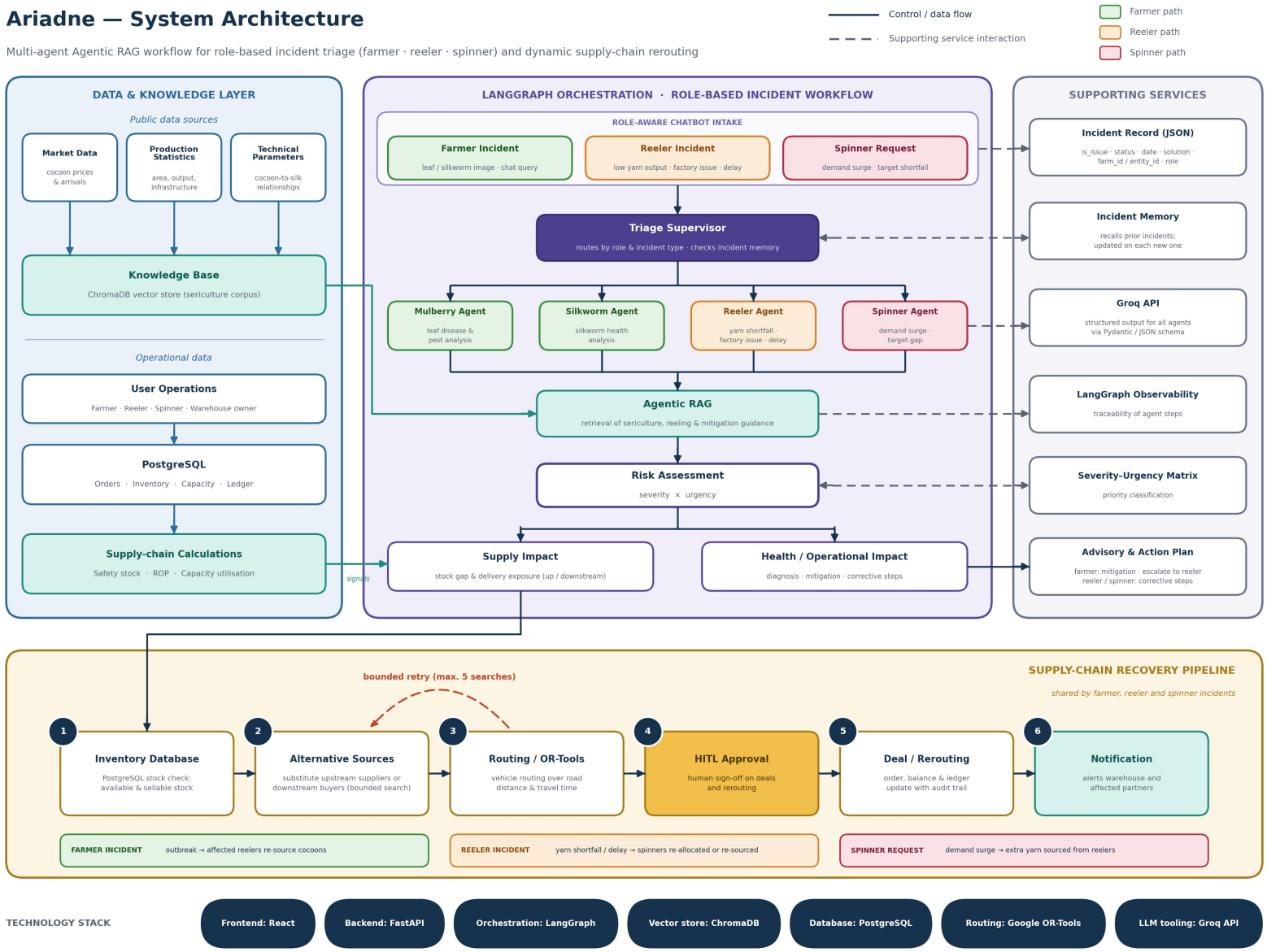


Figure 1. Ariadne system architecture: role-aware incident intake (farmer, reeler, spinner), LangGraph orchestration, data and knowledge layer, and the shared recovery pipeline.

2.2 Component Description

Component	Responsibility
Role-aware Chatbot Intake	Entry point for all actors: a farmer's image or query, a reeler's report of low yarn output, a factory issue or a delay, or a spinner's demand surge or target shortfall. Each report creates an incident.
Triage Supervisor	Consults incident memory, then routes by role and incident type. All specialist collaboration passes through this supervisory agent.
Specialist Agents	*Mulberry* (leaf disease and pests), *Silkworm* (silkworm health), *Reeler* (yarn shortfall, factory issues, delays; uses stock, degradation time, capacity) and *Spinner* (demand surge, target gap against available stock and reeler capacity).
Agentic RAG	Retrieves disease, rearing, reeling and storage guidance and mitigation advice from the ChromaDB knowledge base.
Risk Assessment	Rates severity and urgency (severity–urgency matrix), then splits into a *Health / Operational Impact* branch (diagnosis, mitigation, corrective steps) and a *Supply Impact* branch.
Inventory Database and Supply-chain Calculations	PostgreSQL stock checks (orders, inventory, capacity, ledger) plus safety stock, reorder point (ROP) and capacity utilisation, which feed the supply-impact assessment.
Alternative Sources	Searches in the direction the incident needs: other suppliers upstream (farmers for a reeler, reelers for a spinner) or other buyers downstream. Bounded to at most 5 searches.
Routing / OR-Tools	Vehicle routing over road distance and travel time between warehouses, farmers, reelers and spinners.
HITL Approval, Deal and Notification	A human approves deals and rerouting before any ledger change; orders, balances and the audit trail are then updated and the warehouse and affected partners notified.
Advisory & Action Plan	Output of the health or operational branch: mitigation and escalation to the reeler for farmers, corrective steps for reelers and spinners.

2.3 Data Flow and Design Controls

1. **Ingestion.** A farmer, reeler or spinner submits an image, report or request. This creates an incident, stored as JSON with `is_issue`, `status`, `date`, `solution` and `farm_id` (the identifier refers to the reeler or spinner for their incidents, with a role field), shareable with the warehouse.
2. **Triage and retrieval.** The supervisor checks incident memory and hands off to the Mulberry, Silkworm, Reeler or Spinner agent, which queries Agentic RAG (ChromaDB) for diagnosis, mitigation or corrective guidance.
3. **Risk assessment.** The incident is rated by severity and urgency and split into a health or operational impact (advisory, including whether a farmer should escalate to the reeler) and a supply impact.
4. **Supply impact.** Stock, safety-stock, ROP and capacity signals from PostgreSQL show how many reelers or spinners are affected and how much stock is at risk.
5. **Recovery.** If supply is at risk, Alternative Sources are searched (bounded loop) in the required direction, and OR-Tools optimises routes.
6. **Approval and execution.** HITL approval gates the deal or reroute; then the ledger is updated and notifications are sent.

A farmer incident can cascade: reduced cocoons may lead the reeler to report a yarn shortfall, starting a reeler incident for downstream spinners; incident memory links such incidents.

2.4 Design Controls

- **Bounded loops:** rerouting searches (farmers, reelers, spinners and others) are limited to a fixed maximum, (for eg. 5).
- **Role-based visibility:** each user sees only the data of his own tier, as set out in Section 1.
- **Incident memory:** recurring issues recall earlier incidents, and memory is updated with every new incident.
- **Observability:** LangGraph provides traceability of agent steps.
- **Structured outputs:** the Groq API with Pydantic/JSON schemas keeps agent outputs machine-validatable.
- **Human oversight:** HITL approval finalises all farmer–reeler–spinner and farmer–farmer deals.

Technology stack. React (frontend), FastAPI (backend), ChromaDB (vector store), PostgreSQL (transactional store), Google OR-Tools (vehicle routing) and LangGraph (agent orchestration).

3 Agentic Pattern Mapping

Each pattern is chosen for a specific need of the problem (Table 2).

Table 2. Selected agentic patterns and their rationale.

Pattern	Where it is applied	Rationale
Multi-agent with supervisor	Triage Supervisor coordinating the Mulberry, Silkworm, Reeler and Spinner agents; all collaboration is supervisor-mediated.	Farm, factory and demand problems need distinct domain expertise; a central supervisor keeps routing and handoffs controlled.
Agentic RAG	Agents query the ChromaDB knowledge base of sericulture and warehouse documents (disease management, rearing and reeling practice, mitigation).	Diagnoses and resolutions must rest on documented guidance, not model memory.
Planning and rerouting	Risk Assessment → Supply Impact → Inventory → Alternative Sources → Routing, shared by all three entry points, with bounded loops (max. 5 searches).	Recovery is a staged plan whether the shortfall is upstream (cocoons) or downstream (yarn); bounding loops prevents runaway iteration.
Tool use	Groq API (structured output), database calls (inventory), OR-Tools (paths and distances).	Stock figures and routes must come from authoritative tools, not generated text.
Human-in-the-loop	Approval gate before every deal or reroute (farmer–reeler, reeler–spinner, farmer–farmer).	Deals change balances and stock, so a human decides before the ledger is updated.
Memory and observability	Incident memory consulted on recurrence and updated after each incident; LangGraph tracing of the agent graph.	Repeated issues reuse earlier context; decisions stay debuggable and auditable.

Alternatives rejected. A single monolithic agent would mix farm, factory and demand reasoning and make routing errors hard to trace. Fully autonomous deals would let a wrong inference change balances, which is why approval stays with a human.

4 Data Sources and Tool Plan

4.1 Data Plan

Table 3 lists each data need, its recommended source and its use; stated dependencies (e.g. dataset availability) are noted.

Table 3. Data sources and their intended use.

No.	Data required	Recommended source	Intended use
1	Mulberry disease and pest images	PaddlePaddle dataset	Train or fine-tune the mulberry vision model (study: six classes, 1,788 original images).
2	Silkworm disease images	Silkworm disease research dataset	Healthy-versus-diseased classifier; confirm download availability before implementation.
3	Disease treatment and sericulture knowledge	Central Silk Board; SILKS; Silk India	Populate ChromaDB with disease management, rearing practice, advisories and mitigation.
4	Sericulture statistics and technical knowledge	International Sericultural Commission	Support on mulberry, silkworm breeds and industry statistics.
5	Cocoon quality, reeling yield, processing capacity, storage	FAO Silk Reeling and Testing Manual	Derive cocoon-to-silk requirements and processing constraints.
6	Farmer population, mulberry area, production, government reeling infrastructure	Ramanagara District Sericulture Department	Establish a real regional supply-chain network and its approximate scale.
7	Cocoon market locations and procedures	Karnataka SILKS marketing portal	Represent market nodes and farmer-to-reeler procurement pathways.
8	Historical cocoon prices and market arrivals	Price Dynamics of Mulberry Cocoons in South India	Analyse price trends and supply variation; obtain underlying numerical records if available.
9	Farm, reeler, spinner and warehouse locations	OpenStreetMap plus registered user locations	Compute real distances; no public source gives a complete verified registry of private participants.
10	Road distance and travel time	Google Routes API or OpenStreetMap routing	Distance and travel-time inputs for OR-Tools.
11	Current inventory, sellable and safety stock	User entries, stock receipts, approved transactions	Authoritative stock balances; reorder points and safety stock from observed demand and lead times.
12	Deals, balances and incident history	Application transactions and incident reports (farmer, reeler, spinner)	HITL approvals, ledger updates, incident memory and audit trails.

4.2 Tool Plan

Table 4. Tools and their roles within the agent graph.

Tool	Role
Groq API	Structured output (free package), validated against Pydantic/JSON schemas.
ChromaDB retrieval	Agentic RAG over sericulture and warehouse documents.
PostgreSQL database calls	Inventory checks; retrieval of orders, stock, capacity and ledger records.
Google OR-Tools with Routes API / OSM	Vehicle routing over road distance and travel time between warehouses, farmers, reelers and spinners.
LangGraph	Orchestrates the supervisor, specialist agents and bounded loops; provides observability.

5 Evaluation and Metrics Plan

Quality is judged on four dimensions: agent workflow behaviour, answer quality, retrieval and grounding, and safe handling of uncertainty. Each metric is computed over a dedicated test set covering all three entry points: farmer incidents, reeler incidents (yarn shortfall or delay) and spinner requests (demand surge).

Table 5. Evaluation metrics, computation and success criteria.

Metric	How to evaluate it	What success means
A. Agent workflow and tool behaviour		
Task Completion Rate	Completed tasks ÷ total test tasks × 100	Agent completes the intended workflow.
Supervisor Routing Accuracy	Correctly routed requests ÷ routing test cases × 100	Requests reach the right specialist (Mulberry, Silkworm, Reeler or Spinner).
Tool Selection Accuracy	Correct tool calls ÷ tool-selection test cases × 100	Right database, RAG, routing or other tool is chosen.
Structured Output Validity	Outputs passing schema validation ÷ outputs tested × 100	Outputs conform to Pydantic/JSON schemas.
Tool Execution Success Rate	Successful valid executions ÷ attempted valid executions × 100	Tool calls succeed when preconditions hold.
Handoff Accuracy	Correct specialist handoffs ÷ handoffs evaluated × 100	Context reaches the correct agent intact.
Unnecessary Tool Call Rate	Unnecessary tool calls ÷ total tool calls × 100	No redundant calls or needless loops.
Workflow Termination Accuracy	Correctly completed, rejected or escalated workflows ÷ workflows tested × 100	Graph ends or pauses at the right point.
B. Answer quality		
Answer Correctness	Correct answers ÷ factual questions tested × 100	Responses agree with verified facts.
Completeness	Judge score against a checklist of required answer elements	Covers the important parts of the request.
Relevance	LLM judge scores how directly the response addresses the request	Avoids irrelevant content.
C. Retrieval and grounding		
Groundedness / Faithfulness	Claims supported by retrieved documents ÷ claims checked × 100	No unsupported claims from retrieved sources.
Citation Accuracy	Correctly supported citations ÷ citations evaluated × 100	Citations support their claims.
Retrieval Recall@K	Relevant documents in top K ÷ all known relevant documents × 100	RAG retrieves the relevant source material.
Context Precision@K	Relevant documents in top K ÷ all documents in top K	Little irrelevant context retrieved.
D. Safe handling of uncertainty		
Abstention Accuracy	Correct abstentions or escalations ÷ cases requiring them × 100	Admits uncertainty or seeks human help when evidence is thin.
Hallucination Rate	Unsupported factual claims ÷ factual claims evaluated × 100	Unsupported assertions are minimised.

How the plan is run. (1) *Test sets:* scripted scenarios for each entry point, including edge cases such as zero stock, no alternative source within the 5-search bound, a rejected deal, and a question the knowledge base cannot answer. (2) *Workflow metrics* (section A) are scored from LangGraph traces, comparing the recorded routing, tool calls and termination point with the expected ones. (3) *Answer metrics* (section B) use verified reference answers and a checklist-based LLM judge. (4) *Grounding metrics* (section C) check each claim and citation against the retrieved chunks, with retrieval scored against hand-labelled relevant documents. (5) *Uncertainty metrics* (section D) use cases built to have insufficient evidence, where the correct behaviour is to abstain or escalate to a human. Target thresholds are fixed after a first baseline run.